EAST WEST UNIVERSITY

CSE360: Computer Architecture (Section No.01)

[Summer 2026]

Project Report

- Traffic Control System with Adaptive Control for Vehicles and Pedestrians

Submitted By:

Student Name	Student Id
Nurtaz Reza	2023-3-60-282
Fabiha Shajjad	2023-3-60-398
Tanvir Tahsin	2023-3-60-385
Mohtasim Haque	2023-3-60-439

Submitted To: Nawab Yousuf Ali

Date Of Submission: 19.07.2026

Table of Contents

1. Title: Traffic Control System with adaptive control for vehicles and pedestrians 3
2. Objective 3
3. Theory 4
3.1 Traffic Signal Control 4
3.2 Adaptive Traffic Signal Timing 4
3.3 Traffic Priority 6
3.4 Emergency Vehicle Handling 7
3.5 Multiple Emergency Directions 8
3.6 Pedestrian Crossing 9
3.7 Signal Timing 9
4. Design 10
4.1 System Overview 10
4.2 System Algorithm Flow 12
4.3 Data Structure Design 13
4.4 Input Validation Design 14
5. Implementation 14
5.1 Programming Language 14
5.2 System Requirements 15
5.3 Functional Modules 16
5.4 Hierarchical Relationship 20
6. Debugging-Test-Run 21
6.1 Test Case 1: Normal Traffic 21
6.2 Test Case 2: Heavy Traffic 22
6.3 Test Case 3: Emergency Vehicle 24
6.4 Test Case 4: Multiple Emergency Vehicles 25
6.5 Test Case 5: Multiple Emergency Directions 26
6.6 Test Case 6: Pedestrian Crossing 28
6.7 Test Case 7: Invalid Vehicle Input 28
6.8 Test Case 8: Invalid Menu Input 29
6.9 Debugging Approach 29
7. Results Analysis 30
7.1 Time Complexity 30
7.2 Space Complexity 30
7.3 Robustness 30

7.4 Advantages 31
7.5 Limitations 31
8. Conclusion and Future Improvements 32
8.1 Conclusion 32
8.2 Future Improvements 32
9. Bibliography 33

1. Title: Traffic Control System with adaptive control for vehicles and pedestrians

2. Objective

The main objective of this mini project is to design and implement a computerized traffic signal control system that manages traffic at a four-way intersection based on the number of vehicles present on each road.

The major objectives of the project are:

1. To simulate a four-way traffic intersection using the C programming language.
2. To accept vehicle counts for North, South, East, and West directions.
3. To calculate green-light duration according to traffic density.
4. To prioritize roads having higher numbers of vehicles.
5. To provide special handling for emergency vehicles.
6. To support emergency vehicles approaching from multiple directions.
7. To simulate pedestrian crossing requests.
8. To provide countdown timers for traffic signals.
9. To validate user inputs and prevent invalid values.
10. To maintain and display traffic statistics for completed cycles.
11. To demonstrate how programming concepts can be used to model a real-world control system.

Thus, this project provides a software-based simulation of an adaptive traffic signal controller rather than a physical traffic-light system.

3. Theory

3.1 Traffic Signal Control

A traffic signal controls the movement of vehicles and pedestrians at an intersection. A conventional signal normally consists of three states:

- **Green:** Vehicles are allowed to proceed.
- **Yellow:** Vehicles should prepare to stop.
- **Red:** Vehicles must stop.

In this project, each road is processed individually. During its processing cycle, the corresponding direction receives a green signal, followed by a yellow signal and then a red signal.

The program represents four directions:

- **North**
- **South**
- **East**
- **West**

The signal status is displayed according to the currently active direction. For example, when the North direction is active, North becomes GREEN while the remaining directions become RED.

3.2 Adaptive Traffic Signal Timing

The main idea of the project is to change the green-light duration according to the number of vehicles.

The program uses the following rule:

Number of Vehicles	Traffic Level	Green Time
0–5	Light	5 seconds
6–15	Medium	10 seconds
More than 15	Heavy	15 seconds

The mathematical representation can be written as:

$$G(V) = \begin{cases} 5, & V \leq 5 \\ 10, & 6 \leq V \leq 15 \\ 15, & V > 15 \end{cases}$$

where:

- (V) = number of vehicles
- (G(V)) = assigned green-light duration

This function is implemented by the **calculateGreenTime()** function in the program.

```
int calculateGreenTime(int vehicles) {
    // Adaptive timing based on vehicle count
    if(vehicles <= 5)
        return 5;   // Light traffic
    else if(vehicles <= 15)
        return 10;  // Medium traffic
    else
        return 15;  // Heavy traffic
}
```

Adaptive green-time calculation for normal traffic

For example:

If North has 4 vehicles:

G(4)=5

If South has 12 vehicles:

G(12)= 10

If East has 25 vehicles:

G(25)=15

Therefore, roads with heavier traffic receive longer green-light durations.

3.3 Traffic Priority

After vehicle counts are entered, the four roads are represented using a Road structure.

The **structure** contains:

- Road name
- Number of vehicles
- Direction
- Green-light duration

The program then sorts the four roads according to vehicle count in descending order. The road with the highest number of vehicles receives the highest priority.

The comparison function used for sorting is:

[Priority(A) > Priority(B)]

when

[No. of Vehicles_A > No. of Vehicles_B]

The C library function **qsort()** is used to perform the sorting.

For example:

Direction	Vehicles	Priority
North	25	1
East	18	2
West	10	3
South	4	4

The system then processes the roads in this order.

3.4 Emergency Vehicle Handling

The project allows the user to indicate whether emergency vehicles are present. If emergency vehicles are detected, the system enters its emergency-processing phase before starting the normal traffic cycle.

The emergency green time depends on the number of emergency vehicles:

Emergency Vehicles	Priority Level	Green Time
1	Standard	8 seconds
2–3	High	12 seconds
4 or more	Critical	15 seconds

The formula is:

$$E(C) = \begin{cases} 8, & C = 1 \\ 12, & 2 \le C \le 3 \\ 15, & C \ge 4 \end{cases}$$

where (C) represents the number of emergency vehicles.

This calculation is implemented in calculateEmergencyGreenTime().

```
int calculateEmergencyGreenTime(int emergencyCount) {
    // Adaptive timing based on number of emergency vehicles
    if(emergencyCount == 1)
        return 8;   // Single emergency vehicle
    else if(emergencyCount <= 3)
        return 12;  // 2-3 emergency vehicles
    else
        return 15;  // 4+ emergency vehicles (major incident)
}
```

Adaptive green-time calculation for emergency vehicles

3.5 Multiple Emergency Directions

The system can process emergency vehicles from more than one direction. The user can specify between one and four directions containing emergency vehicles. The program uses an array called **usedDirections** to ensure that the same direction is not selected twice during one emergency-processing operation.

For each selected direction, the program:

1. Identifies the direction.
2. Receives the number of emergency vehicles.
3. Calculates the appropriate emergency green time.
4. Displays the emergency priority level.
5. Activates the emergency green signal.
6. Runs the countdown timer.
7. Changes the signal to yellow.
8. Reports that the emergency vehicles have passed.

3.6 Pedestrian Crossing

The system also provides a pedestrian crossing operation. When a pedestrian crossing request is received, all vehicle signals are changed to RED and the pedestrian signal becomes GREEN. The pedestrian crossing timer is set to:

$T_p = 7$

The implementation activates all vehicle signals as RED, displays the pedestrian signal as GREEN and runs a seven-second countdown.

3.7 Signal Timing

For normal traffic, the complete signal processing for a road consists of:

$$T_{\{normal\}} = T_G + T_Y + T_R$$

where:

- (T_G) = adaptive green time
- (T_Y) = yellow time
- (T_R) = additional red-state delay

The yellow signal is fixed at:

$T_Y = 3$

The program uses a real one-second delay during countdown operations through the Windows Sleep() function.

---

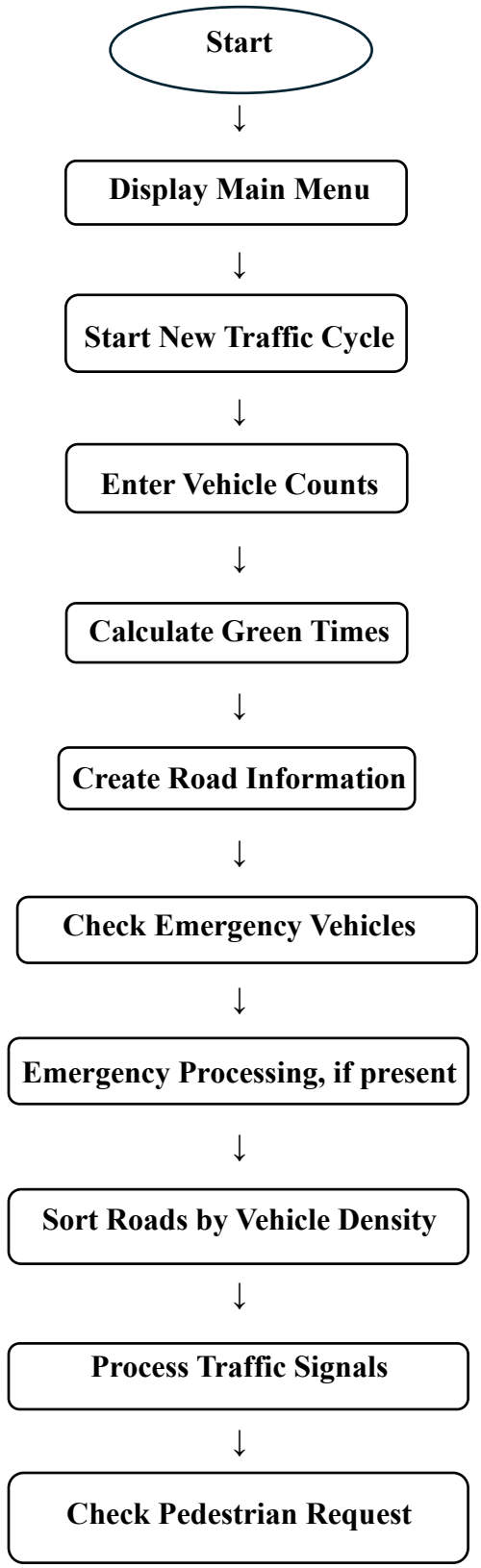
4. Design

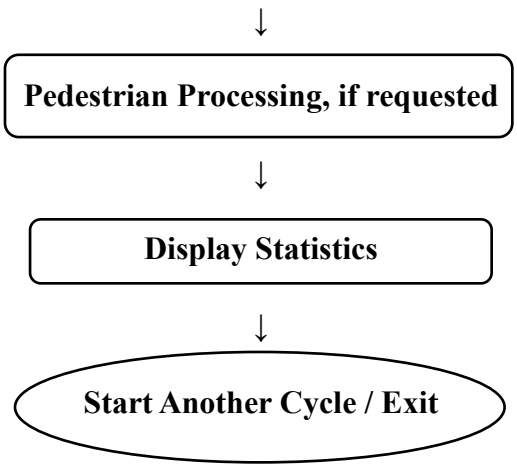
4.1 System Overview

The system can be divided into the following functional modules:

1. Main Menu Module
2. Input Validation Module
3. Vehicle Density Module
4. Adaptive Green-Time Module
5. Traffic Priority Module
6. Traffic Signal Module
7. Emergency Vehicle Module
8. Pedestrian Crossing Module
9. Statistics Module
10. Console Display Module

The overall system follows this general process:





4.2 System Algorithm Flow

The traffic-cycle algorithm can be described as follows:

Algorithm

Step 1: Start the program.

Step 2: Display the main menu.

Step 3: Ask the user whether to start a new traffic cycle or exit.

Step 4: If the user chooses a new cycle, receive vehicle counts for all four directions.

Step 5: Validate each vehicle count.

Step 6: Calculate adaptive green time for each direction.

Step 7: Store the information of each direction in a Road structure.

Step 8: Ask whether emergency vehicles are present.

Step 9: If emergency vehicles are present, process emergency directions.

Step 10: Sort all four roads according to vehicle count.

Step 11: Process each road according to the calculated priority.

Step 12: Ask whether pedestrian crossing is required.

Step 13: If requested, activate the pedestrian crossing.

Step 14: Display current and cumulative traffic statistics.

Step 15: Repeat or exit.

4.3 Data Structure Design

The project uses a structure named **Road**.

The structure can be represented as:

Field	Data Type	Purpose
name	char[10]	Stores road name
vehicles	int	Stores vehicle count
direction	int	Stores direction number
greenTime	int	Stores calculated green time

```
typedef struct {
    char name[10];
    int vehicles;
    int direction;  // 1=North, 2=South, 3=East, 4=West
    int greenTime;
} Road;
```

Road Data Structure

The structure is defined in the program as a collection of information representing one traffic direction.

Four instances are created:

- roads[0] → North
- roads[1] → South
- roads[2] → East
- roads[3] → West

4.4 Input Validation Design

Input validation is an important part of the system because incorrect input could cause incorrect traffic processing.

The program includes separate validation functions for:

- General integer input
- Yes/No input
- Vehicle count input

The general validation function rejects non-numeric input and values outside the specified range.

Vehicle counts are also prevented from being negative.

5. Implementation

5.1 Programming Language

The project is implemented using:

Programming Language: C

The program uses standard C libraries including:

#include <stdio.h>

#include <stdlib.h>

#include <time.h>

#include <string.h>

It also uses:

#include <windows.h>

for Windows-specific console functionality and real-time delays.

5.2 System Requirements

Hardware Requirements

The project does not require specialized hardware because it is a software simulation.

Minimum requirements include:

- Processor: Any modern x86/x64 processor
- RAM: 2 GB or higher
- Storage: 200 MB of free space
- Keyboard and monitor

Software Requirements

- Windows operating system
- C compiler
- IDE or text editor
- Command-line terminal/console

Examples of suitable development environments include:

- Code::Blocks
- Dev-C++
- Visual Studio
- Visual Studio Code with a C compiler

Because the program uses windows.h, Sleep(), GetStdHandle(), SetConsoleTextAttribute(), system("cls"), and getch(), a Windows-compatible environment is required unless these components are replaced with platform-independent alternatives.

5.3 Functional Modules

Module 1: Main Menu

The main menu allows the user to:

1. Start a new traffic cycle.
2. Exit the system.

The choice is validated before execution.

Module 2: Vehicle Input

The system receives vehicle counts for:

- North
- South
- East
- West

Negative vehicle counts are rejected.

```
int getVehicleCount(char* direction) {
    int count;
    int valid = 0;

    while(!valid) {
        printf("  %s: ", direction);

        if(scanf("%d", &count) != 1) {
            // Clear input buffer
            while(getchar() != '\n');
            printf("\n  Error: Please enter a valid number!\n");
        } else if(count < 0) {
            printf("\n  Error: Vehicle no. cannot be negative!\n");
        } else {
            valid = 1;
        }
    }
```

Vehicle-count Input Validation

Module 3: Adaptive Timing

The **calculateGreenTime()** function determines green duration based on traffic density.

This allows the system to simulate different traffic conditions.

Module 4: Signal Processing

The **processTrafficCycle()** function controls the normal signal sequence:

GREEN → YELLOW → RED

```
void processTrafficCycle(int road, int greenTime, char* roadName, int vehicles) {
    displaySignalStatus(road);

    printf("\n  Direction: %s | Vehicles: %d | Green Time: %d sec\n", roadName,
vehicles, greenTime);

    setColor(GREEN_TEXT);
    printf("\n   %s: GREEN Signal Active\n", roadName);
    printf("  Green Timer: ");
    countdown(greenTime);
    setColor(WHITE_TEXT);

    setColor(YELLOW_TEXT);
    printf("\n   %s: YELLOW Signal Active\n", roadName);
    printf("  Yellow Timer: ");
    countdown(3);
    setColor(WHITE_TEXT);

    setColor(RED_TEXT);
    printf("\n   %s: RED Signal Active\n", roadName);
    setColor(WHITE_TEXT);
```

Normal Traffic Signal Processing

It displays the direction, vehicle count, green time, and countdown timers.

Module 5: Priority Processing

The **runPriorityBasedTrafficCycle()** function sorts the four roads by vehicle density and processes them in priority order.

```
int compareRoads(const void* a, const void* b) {
    Road* roadA = (Road*)a;
    Road* roadB = (Road*)b;
    return roadB->vehicles - roadA->vehicles;

void runPriorityBasedTrafficCycle(Road roads[]) {

    qsort(roads, 4, sizeof(Road), compareRoads);

    printf("\n  Traffic Priority Order:\n");

    for(int i = 0; i < 4; i++) {
        printf("  %d. %s - %d vehicles (Green: %d sec)\n",
               i+1, roads[i].name, roads[i].vehicles, roads[i].greenTime);
    }
```

Priority-based Sorting and Traffic Processing

Module 6: Emergency Processing

The emergency module supports emergency vehicles from multiple directions.

It calculates emergency green time and displays the corresponding priority level.

```
int usedDirections[4] = {0};

for(int i = 0; i < numEmergencyDirections; i++) {
    int validDirection = 0;

    while(!validDirection) {
        direction = getValidInput(1, 4, "Enter Choice: ");

        if(usedDirections[direction - 1]) {
            printf("Direction already selected!\n");
        }
```

```
        else {
            usedDirections[direction - 1] = 1;
            validDirection = 1;
        }
    }

    emergencyCount = getValidInput(1, 100, "Enter: ");

    processSingleEmergency(
        direction,
        emergencyCount,
        dirName[direction]
    );
}
```

Multiple Emergency-direction Handling

Module 7: Pedestrian Processing

The pedestrian module temporarily stops vehicle movement and provides a seven-second pedestrian crossing interval.

```
void handlePedestrian() {
    printf("\nAll Vehicle Signals: ");
    setColor(RED_TEXT);
    printf("RED\n");

    setColor(WHITE_TEXT);
    printf("Pedestrian Signal: ");

    setColor(GREEN_TEXT);
    printf("GREEN\n");

    printf("\nPedestrian Crossing Timer: ");
    countdown(7);

    setColor(WHITE_TEXT);
}
```

Pedestrian Crossing Control

Module 8: Statistics

The system maintains:

- Total normal vehicles
- Total emergency vehicles
- Grand total vehicles
- Total completed traffic cycles

The statistics module displays both current-cycle and cumulative information.

5.4 Hierarchical Relationship

The program can be represented hierarchically as:

main()

- displayMenu()
    - getValidInput()
- getVehicleCount()
- calculateGreenTime()
- getYesNoInput()
    - getValidInput()
- handleEmergency()
    - getValidInput()
    - processSingleEmergency()
        - calculateEmergencyGreenTime()
        - displaySignalStatus()
        - countdown()

- runPriorityBasedTrafficCycle()
    - compareRoads()
    - processTrafficCycle()
        - displaySignalStatus()
        - countdown()
- handlePedestrian()
    - countdown()
- displayStatistics()

This hierarchical structure makes the program easier to understand and separates different traffic-control operations into individual functions.

---

6. Debugging-Test-Run

Testing was performed by providing different types of inputs to verify whether the system behaves correctly.

6.1 Test Case 1: Normal Traffic

Input:

Direction	Vehicles
North	5
South	10
East	20
West	3

Expected result:

```
Traffic Priority Order:
1. EAST - 20 vehicles (Green: 15 sec)
2. SOUTH - 10 vehicles (Green: 10 sec)
3. NORTH - 5 vehicles (Green: 5 sec)
4. WEST - 3 vehicles (Green: 5 sec)


[PRIORITY 1/4] Processing EAST Direction

----- TRAFFIC SIGNAL STATUS -----

North : RED
South : RED
East  : GREEN
West  : RED
```

6.2 Test Case 2: Heavy Traffic

Input:

Direction	Vehicles
North	30
South	25
East	20
West	18

All four directions should receive 15 seconds of green time.

Expected result:

North:

```
[PRIORITY 1/4] Processing NORTH Direction

----- TRAFFIC SIGNAL STATUS -----

North : GREEN
South : RED
East  : RED
West  : RED

Direction: NORTH | Vehicles: 30 | Green Time: 15 sec

 NORTH: GREEN Signal Active
Green Timer: 15 14 13 12 11 10  9  8  7  6  5  4  3  2  1
```

South:

```
[PRIORITY 2/4] Processing SOUTH Direction

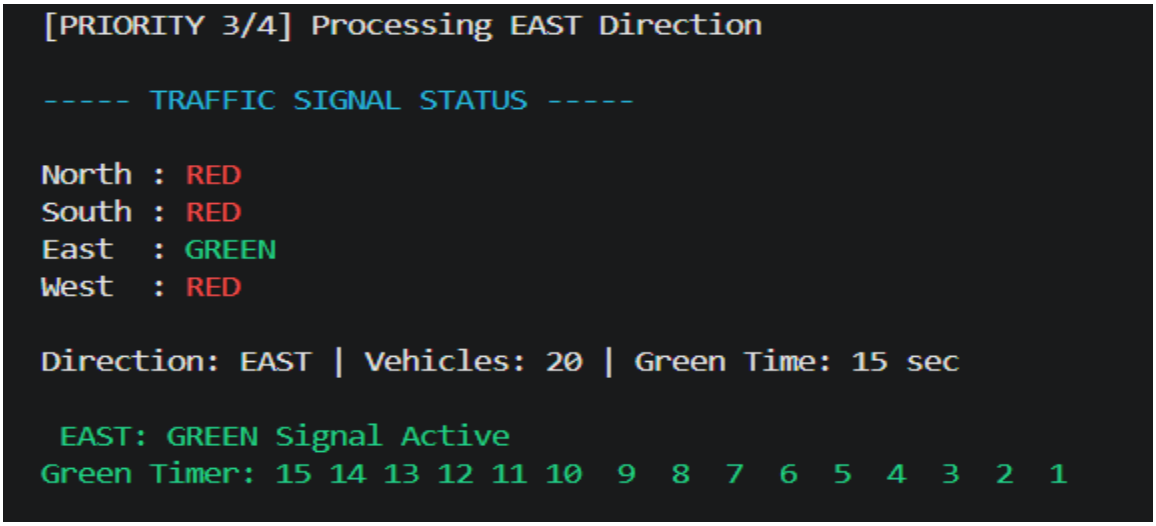
----- TRAFFIC SIGNAL STATUS -----

North : RED
South : GREEN
East  : RED
West  : RED

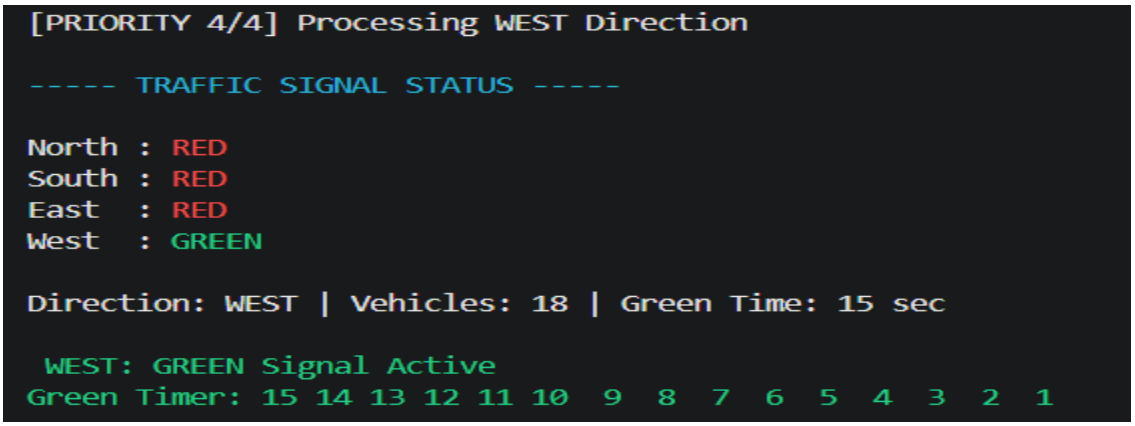
Direction: SOUTH | Vehicles: 25 | Green Time: 15 sec

 SOUTH: GREEN Signal Active
Green Timer: 15 14 13 12 11 10  9  8  7  6  5  4  3  2  1
```

East:



West:



6.3 Test Case 3: Emergency Vehicle

Input:

Emergency vehicle present: Yes

Emergency direction: North

Emergency vehicles: 1

Expected result:

```
Clearing EMERGENCY LANE for NORTH direction...

Emergency Details:

Direction          : NORTH
Emergency Vehicles : 1
Adaptive Green Time: 8 seconds

Priority Level     : Standard (Single Vehicle)

----- TRAFFIC SIGNAL STATUS -----

North : GREEN
South : RED
East  : RED
West  : RED

EMERGENCY LANE: NORTH
Emergency GREEN Signal Active
Timer:  8  7  6  5  4  3  2  1
```

6.4 Test Case 4: Multiple Emergency Vehicles

Input:

Emergency direction: East

Emergency vehicles: 5

Expected result:

```
Emergency Details:

Direction          : EAST
Emergency Vehicles : 5
Adaptive Green Time: 15 seconds

Priority Level     : CRITICAL (Major Incident)

----- TRAFFIC SIGNAL STATUS -----

North : RED
South : RED
East  : GREEN
West  : RED

EMERGENCY LANE: EAST
Emergency GREEN Signal Active
Timer: 15 14 13 12 11 10  9  8  7  6  5  4  3  2  1
```

6.5 Test Case 5: Multiple Emergency Directions

The program allows emergency vehicles to be entered for multiple directions.

For example:

Direction	Emergency Vehicles
North	2
East	4

Expected result:

```
Emergency Details:

Direction           : EAST
Emergency Vehicles  : 4
Adaptive Green Time: 15 seconds

Priority Level      : CRITICAL (Major Incident)

----- TRAFFIC SIGNAL STATUS -----

North : RED
South : RED
East  : GREEN
West  : RED

EMERGENCY LANE: EAST
Emergency GREEN Signal Active
Timer: 15 14 13 12 11 10  9  8  7  6  5  4  3  2  1

Yellow Signal:
Timer:  3  2  1

4 Emergency Vehicle(s) Passed Successfully on EAST!
```

```
Emergency Details:

Direction           : NORTH
Emergency Vehicles  : 2
Adaptive Green Time: 12 seconds

Priority Level      : High (Multiple Vehicles)

----- TRAFFIC SIGNAL STATUS -----

North : GREEN
South : RED
East  : RED
West  : RED

EMERGENCY LANE: NORTH
Emergency GREEN Signal Active
Timer: 12 11 10  9  8  7  6  5  4  3  2  1

Yellow Signal:
Timer:  3  2  1

2 Emergency Vehicle(s) Passed Successfully on NORTH!
 Emergency Processing Complete!
Total Emergency Directions: 2
Total Emergency Vehicles  : 6
```

6.6 Test Case 6: Pedestrian Crossing

Input:

Pedestrian crossing request: Yes

Expected result:

```
Pedestrian Crossing Request?
1 = Yes | 0 = No
Enter: 1

=====================================
   PEDESTRIAN CROSSING ACTIVATED
=====================================

All Vehicle Signals: RED
Pedestrian Signal  : GREEN

Pedestrian Crossing Timer:  7  6  5  4  3  2  1

Pedestrian Crossing Completed
```

6.7 Test Case 7: Invalid Vehicle Input

Input:

North: -5

Expected result:

```
Enter Vehicle No. for Each Direction:

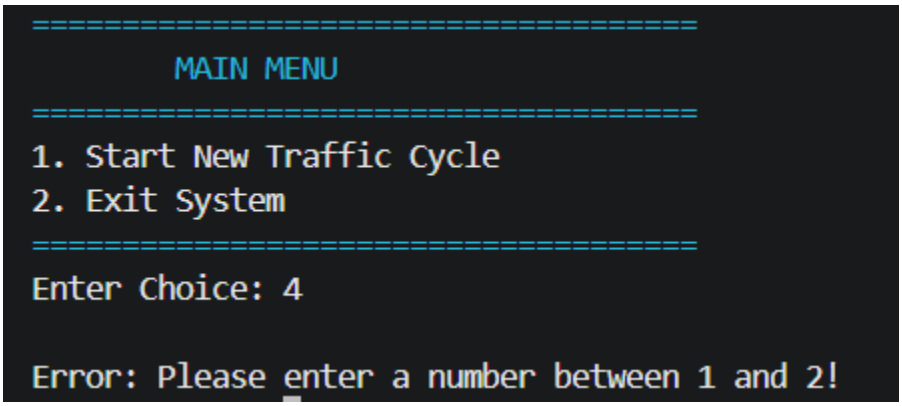
North: -5

Error: Vehicle no. cannot be negative!
North:
```

6.8 Test Case 8: Invalid Menu Input

Input:

Enter Choice: 4



6.9 Debugging Approach

The following debugging approaches were used:

1. **Input validation** to prevent invalid numeric values.
2. **Range checking** for menu choices and emergency directions.
3. **Duplicate direction checking** during emergency processing.
4. **Function decomposition** to isolate individual operations.
5. **Manual test cases** for different traffic densities.
6. **Boundary testing** around traffic thresholds.
7. **Testing emergency conditions** with different vehicle counts.
8. **Testing pedestrian requests** separately from normal traffic.

Special attention was given to boundary values such as 5, 6, 15, and 16 vehicles because these values determine changes in green-light duration.

---

7. Results Analysis

7.1 Time Complexity

The most significant algorithmic operation in the system is sorting the four roads according to vehicle density. The program uses the standard C library function qsort() to perform this sorting.

The C standard defines the behavior of qsort() but does not require a particular sorting algorithm to be used internally. Therefore, its exact time complexity depends on the library implementation.

For (n) roads, the average and worst-case complexity of a comparison-based sorting operation such as quicksort can be represented approximately as:

Average case = O(n log n)

Worst case = $O(n^2)$

However, in this project:

n = 4, because the system always processes exactly four directions.

Therefore, the **time complexity** = O(1)

7.2 Space Complexity

The system stores four Road structures. So, the space used is a constant

Therefore, **space complexity** = O(1)

7.3 Robustness

The program demonstrates robustness through input validation.

For example, vehicle counts cannot be negative, and invalid numeric input is rejected. The emergency module also prevents selecting the same direction more than once during a single

emergency operation. This reduces the possibility of incorrect program execution caused by invalid user input.

7.4 Advantages

The major advantages of the proposed system are:

1. Adaptive green-light duration.
2. Traffic-density-based priority.
3. Emergency vehicle handling.
4. Support for multiple emergency directions.
5. Pedestrian crossing simulation.
6. Input validation.
7. Real-time countdown simulation.
8. Traffic statistics.
9. Modular function-based design.
10. Simple console-based implementation.

7.5 Limitations

The current implementation is a simulation and has several limitations.

1. Manual Vehicle Input

2. Fixed Four-Direction Intersection

3. Fixed Timing Categories

4. No Physical Hardware

5. Windows Dependency

6. No Persistent Database

7. Simplified Traffic Model

Real-world traffic conditions are much more complex than vehicle count alone. Factors such as lane capacity, road length, vehicle speed, turning movements, and traffic flow rates are not modeled in the current implementation.

---

8. Conclusion and Future Improvements

8.1 Conclusion

The **Priority-Based Adaptive Traffic Signal Control System** successfully demonstrates how a computer program can simulate intelligent traffic signal management.

The system accepts vehicle counts from four directions and uses these values to determine adaptive green-light durations. Roads with higher vehicle densities receive higher processing priority. The program also provides special handling for emergency vehicles and supports emergency vehicles from multiple directions.

In addition, the system provides pedestrian crossing functionality, countdown timers, input validation, and cumulative traffic statistics.

The project demonstrates several important programming and computer-system concepts, including data structures, functions, conditional statements, sorting algorithms, input validation, timing operations, and modular software design.

Although the project is a software simulation rather than a physical traffic control system, it provides a foundation that could be expanded into a more realistic traffic management solution.

8.2 Future Improvements

Several improvements could be introduced in future versions.

1. Automatic Vehicle Detection

2. Real-Time Sensor Integration

3. Dynamic Timing

4. Physical Traffic Signal Integration

5. Data Storage

6. Graphical User Interface

7. Larger Traffic Networks

8. More Detailed Traffic Modeling

---

9. Bibliography

1. McElroy, R. (2008). Traffic Signal Timing Manual. In *U.S. Department of Transportation Federal Highway Administration* (p. 265) [Report]. U.S. Department of Transportation.
2. *c programming language | Pearson US*. (n.d.). Pearson.
3. cppreference.com. (n.d.). qsort. C algorithm library.
4. Karl-Bridge-Microsoft. (n.d.). *Sleep function (synchapi.h) - Win32 apps*. Microsoft Learn.
5. Miniksa. (n.d.). *GetStdHandle function - Windows Console*. Microsoft Learn.
6. Miniksa. (n.d.-b). *SetConsoleTextAttribute function - Windows Console*. Microsoft Learn.